

[Learning Objectives](#)

[Part 1: Introduction to IT Operations & Infrastructure Context \(25 min\)](#)

[1.1 What Is IT Operations?](#)

[1.2 Why “Infrastructure Context” Comes First](#)

[1.3 The Modern IT Landscape](#)

[1.4 Discussion Prompt \(for class\)](#)

[Part 2: Configuration Management Database \(CMDB\) \(25 min\)](#)

[2.1 What Is a CMDB?](#)

[2.2 Why CMDBs Exist](#)

[2.3 CMDB Data Model \(Core Concepts\)](#)

[2.4 How CMDBs Are Populated](#)

[2.5 CMDB in Practice — Common Tools](#)

[2.6 Class Exercise \(5 min\)](#)

[Part 3: Topology Mapping \(25 min\)](#)

[3.1 What Is Topology Mapping?](#)

[3.2 Types of Topology](#)

[3.3 Why Topology Mapping Matters for Operations](#)

[3.4 Static vs. Dynamic Topology](#)

[3.5 How Topology Is Discovered](#)

[3.6 Topology + Incident Correlation](#)

[3.7 Class Exercise \(5 min\)](#)

[Part 4: Metrics, Logs & Traces — The Three Pillars of Observability \(35 min\)](#)

[4.1 What Is Observability?](#)

[4.2 Pillar 1 — Metrics](#)

[4.3 Pillar 2 — Logs](#)

[4.4 Pillar 3 — Traces](#)

[4.5 Comparing the Three Pillars](#)

[4.6 How the Pillars Work Together \(Worked Example\)](#)

[4.7 Class Exercise \(10 min\)](#)

[Part 5: Recap & Discussion \(10 min\)](#)

[Key Takeaways](#)

[Discussion Questions for Wrap-Up](#)

[Suggested Pre-Reading for Lecture 2](#)

Learning Objectives

By the end of this session, students will be able to:

1. Explain what modern IT Operations (ITOps) covers and why “infrastructure context” matters before any monitoring or AI tool is introduced.
2. Describe the role of a Configuration Management Database (CMDB) in an enterprise environment.
3. Understand topology mapping and how dependency maps are built and used during incidents.
4. Define the three pillars of observability — Metrics, Logs, and Traces — and explain how they differ and complement each other.
5. Connect these foundational concepts to real-world outage scenarios.

Session Time Allocation (120 minutes)

Segment	Topic	Duration
---------	-------	----------

1	Introduction to IT Operations & Infrastructure Context	25 min
2	Configuration Management Database (CMDB)	25 min
3	Topology Mapping	25 min
4	Metrics, Logs & Traces (Observability Pillars)	35 min
5	Recap, Discussion & Q&A	10 min

Part 1: Introduction to IT Operations & Infrastructure Context (25 min)

1.1 What Is IT Operations?

IT Operations (ITOps) is the set of processes, tools, and teams responsible for keeping an organization's technology environment running: servers, networks, databases, applications, and cloud services. ITOps teams are responsible for:

- **Availability** — keeping systems up and reachable.
- **Performance** — ensuring acceptable response times and throughput.
- **Change management** — safely rolling out updates without breaking production.
- **Incident response** — detecting, diagnosing, and resolving outages quickly.
- **Capacity planning** — making sure infrastructure scales with demand.

Traditionally, ITOps was reactive: something breaks, a ticket is raised, an engineer investigates. Modern ITOps is shifting toward **proactive and predictive operations**, largely enabled by AI/ML (which we cover in Lecture 2) — but none of that AI works without a solid foundation of *context*. That context is what this lecture is about.

1.2 Why “Infrastructure Context” Comes First

Imagine an alert fires: “CPU usage on server X is at 95%.” Without context, this is just a number. With context, an engineer instantly knows:

- What application runs on server X
- Which business service depends on it (e.g., “Online Checkout”)
- Who owns it (which team to page)
- What else is connected to it (database, load balancer, downstream APIs)
- Whether this server is critical (production) or non-critical (dev/test)

This surrounding information — the *infrastructure context* — is what turns a raw signal into an actionable insight. Without it, operations teams “chase alerts” instead of solving problems.

Key idea: Observability tools generate *data*. Infrastructure context turns that data into *meaning*.

1.3 The Modern IT Landscape

Today's infrastructure context is far more complex than a decade ago:

- **On-premises** data centers (physical servers, storage arrays, network switches)
- **Private and public cloud** (AWS, Azure, GCP) — elastic, ephemeral resources
- **Containers & orchestration** (Docker, Kubernetes/OpenShift) — workloads that move across nodes
- **Microservices** — a single “application” may be 50+ independently deployed services
- **Hybrid and multi-cloud** — mixtures of all the above

This complexity is exactly why context and mapping (CMDB, topology) have become *more* important, not less — a human can no longer hold the entire environment “in their head.”

1.4 Discussion Prompt (for class)

"If a payment service goes down at 2 AM, what information would you want on your screen in the first 60 seconds?" — Let students brainstorm before revealing that this list maps almost exactly to CMDB + topology + observability data covered in this lecture.

Part 2: Configuration Management Database (CMDB) (25 min)

2.1 What Is a CMDB?

A **Configuration Management Database (CMDB)** is a centralized repository that stores information about all the **Configuration Items (CIs)** in an IT environment and the relationships between them.

A **Configuration Item (CI)** is any component that needs to be managed to deliver an IT service — examples include:

- Physical servers, virtual machines, containers
- Network devices (routers, switches, load balancers)
- Applications and software packages
- Databases
- Business services (e.g., "Customer Portal")
- Documentation and contracts (in some frameworks)

2.2 Why CMDBs Exist

Without a CMDB, organizations rely on tribal knowledge — a small number of senior engineers who "just know" how systems connect. This doesn't scale and creates massive risk (bus factor). A CMDB provides:

1. **A single source of truth** for what exists in the environment.
2. **Impact analysis** — "If I patch this server, what services are affected?"
3. **Faster incident triage** — connecting an alert to the business service it affects.
4. **Change management support** — assessing risk before deployments.
5. **Compliance & audit** — proving what software/hardware exists and who owns it.

2.3 CMDB Data Model (Core Concepts)

Concept	Description	Example
CI (Configuration Item)	A discrete, trackable IT component	web-server-07
CI Class/Type	Category of the CI	Server, Database, Application
Attributes	Properties of a CI	OS version, IP address, owner
Relationship	A link between two CIs	"runs on," "depends on," "hosted by"
CI Owner	Person/team responsible	Platform Team

Example relationship chain:

```
Business Service: Online Checkout
├─ depends on → Application: checkout-api
│   └─ runs on → Server: web-server-07
│       └─ hosted in → Data Center: DC-East
└─ depends on → Database: orders-db-primary
```

This chain is exactly what lets an engineer trace a low-level alert ("server disk full") up to its business impact ("checkout is down").

2.4 How CMDBs Are Populated

- **Manual entry** — historically common, but error-prone and quickly goes stale.
- **Discovery tools** — automated scanners (e.g., ServiceNow Discovery, BMC Discovery) that scan the network and continuously update CI data.
- **Integration/APIs** — cloud providers, Kubernetes clusters, and monitoring tools feed CI data automatically.

Common pitfall: A “stale CMDB” — one that isn’t automatically refreshed — becomes actively dangerous, because teams make decisions based on data that no longer reflects reality. This is one of the most common real-world CMDB failures organizations face.

2.5 CMDB in Practice — Common Tools

- ServiceNow CMDB
- BMC Helix / Atrium CMDB
- Device42
- Cloud-native equivalents: AWS Config, Azure Resource Graph

2.6 Class Exercise (5 min)

Give students a mini scenario: “Server db-node-3 is down.” Ask them to sketch (on paper or whiteboard) what a CMDB relationship chain from that server up to a business service might look like, including at least 3 CI types.

Part 3: Topology Mapping (25 min)

3.1 What Is Topology Mapping?

Topology mapping is the visual and data representation of how components in an IT environment are connected — physically (network topology) and logically (application/service topology).

While a CMDB *stores* relationships as data, topology mapping *visualizes* and often *dynamically discovers* those relationships, especially in fast-changing environments like Kubernetes.

3.2 Types of Topology

1. **Physical/Network Topology** — how devices are physically or network-wise connected (switches, routers, cabling, VLANs).
2. **Application/Service Topology** — how software components call each other (Service A calls Service B calls Database C).
3. **Infrastructure Topology** — how compute, storage, and network resources are provisioned and connected (e.g., which VMs sit on which hypervisor, which pods run on which Kubernetes node).

3.3 Why Topology Mapping Matters for Operations

- **Blast radius analysis** — “If this node fails, what else is affected?”
- **Root cause localization** — tracing an issue upstream/downstream through dependent services.
- **Change risk assessment** — visualizing what a planned change might disrupt.
- **Onboarding** — new engineers understand a complex environment far faster with a visual map than with documents alone.

3.4 Static vs. Dynamic Topology

Aspect	Static Topology	Dynamic Topology
Source	Manually drawn diagrams, CMDB	Auto-discovered from live traffic/APIs

	exports	
Update frequency	Infrequent, often outdated	Near real-time
Best suited for	Stable, traditional infrastructure	Cloud-native, containerized, microservices
Example tool	Visio, static CMDB diagrams	Service mesh maps (Istio), APM topology (Dynatrace, AppDynamics), Kubernetes network graphs

Key teaching point: In modern containerized environments, IP addresses and even pod locations change constantly (pods are ephemeral). Static topology diagrams become obsolete within minutes. This is why dynamic, auto-discovered topology has become essential in cloud-native operations.

3.5 How Topology Is Discovered

- **Network-level discovery** — SNMP, network scanning, flow data (NetFlow).
- **Application-level discovery** — Application Performance Monitoring (APM) agents inject tracing headers to see which services call which.
- **Kubernetes-native discovery** — reading the Kubernetes API (pods, services, ingress objects) directly to build a live map.
- **Service mesh telemetry** — tools like Istio or Linkerd automatically produce a call-graph of every service-to-service request.

3.6 Topology + Incident Correlation

This is the concept that bridges into Lecture 3 (Event Correlation & RCA): when multiple alerts fire simultaneously, topology data lets an AI/automation engine determine which alert is the **root cause** and which are **symptoms**, based on the dependency graph — rather than treating every alert as an independent, equally important event.

Example: - Alert 1: Database CPU high - Alert 2: API response time degraded - Alert 3: Checkout page errors for users

If topology shows *Checkout page* → *API* → *Database*, the engine can infer the database alert is the likely root cause and the other two are downstream symptoms — collapsing 3 alerts into 1 actionable incident.

3.7 Class Exercise (5 min)

Show students three unlabeled diagrams (star, hierarchical, mesh) and ask them to identify which best represents a microservices application versus a traditional 3-tier application versus a full mesh network.

Part 4: Metrics, Logs & Traces — The Three Pillars of Observability (35 min)

4.1 What Is Observability?

Observability is the ability to understand the internal state of a system based on the external data it produces. It goes beyond traditional “monitoring” (which answers *known* questions like “is the server up?”) to help answer *unknown* questions (“why is this specific user’s request slow, and where exactly in the system did it slow down?”).

Observability is commonly described using **three pillars: Metrics, Logs, and Traces**.

4.2 Pillar 1 — Metrics

Definition: Metrics are numeric measurements captured over time, typically aggregated (e.g., average,

sum, percentile) and stored as time-series data.

Examples: - CPU utilization (%) - Memory usage (GB) - Request rate (requests/sec) - Error rate (%) - Latency (p50, p95, p99 in ms)

Characteristics: - Lightweight and efficient to store (good for long retention and dashboards). - Great for trends, thresholds, and alerting (“alert if CPU > 90% for 5 minutes”). - Aggregated — you lose the detail of any *single* event.

Common tools: Prometheus, Grafana, Datadog, CloudWatch.

The four “golden signals” (from Google’s SRE book) — a widely taught framework for what metrics matter most:

Golden Signal	Meaning
Latency	Time taken to service a request
Traffic	Demand on the system (requests/sec)
Errors	Rate of failed requests
Saturation	How “full” the system is (CPU, memory, queue depth)

4.3 Pillar 2 — Logs

Definition: Logs are timestamped, immutable records of discrete events that happened in a system — usually free-text or structured (JSON) messages.

Examples:

```
2026-09-05T10:15:32Z ERROR [checkout-api] OrderService - Failed to process order 88291:
connection timeout to orders-db-primary after 5000ms
```

Types of logs: - **Application logs** — generated by custom application code. - **System logs** — OS-level events (syslog, Windows Event Log). - **Access logs** — web server/API gateway request logs. - **Audit logs** — security and compliance-related events.

Structured vs. Unstructured Logs:

Unstructured	Structured (JSON)
Failed to connect to DB	<pre>{"level": "error", "service": "checkout-api", "event": "db_connect_failed", "db": "orders-db-primary"}</pre>
Hard to query at scale	Easy to filter, aggregate, and correlate

Common tools: ELK Stack (Elasticsearch, Logstash, Kibana), Splunk, Loki, Fluentd.

Strength: Logs give the *detailed narrative* of exactly what happened, including error messages and stack traces — the “ground truth” for root cause investigation.

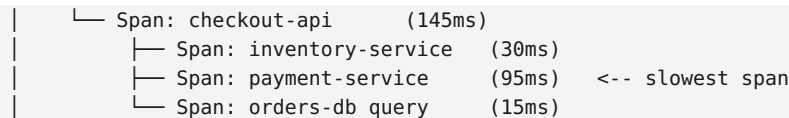
Weakness: High volume — a large system can generate terabytes of logs per day, making raw log storage/search expensive without indexing strategy or sampling.

4.4 Pillar 3 — Traces

Definition: A trace represents the end-to-end journey of a single request as it flows through multiple services in a distributed system. A trace is composed of multiple **spans**, where each span represents a unit of work in one service.

Example — a single checkout request:

```
Trace ID: 7f3a9c...
└─ Span: API Gateway (12ms)
```



This trace instantly shows *where* in a 145ms request the time was spent — in this case, the payment-service call is the bottleneck, not the database.

Why traces matter in microservices: In a monolithic application, you could often find the slow line of code directly. In a system with 50+ microservices, a single user request might touch 10+ services — without tracing, finding the bottleneck is like finding a needle in a haystack.

Common tools: Jaeger, Zipkin, OpenTelemetry, AWS X-Ray.

Key standard to mention: OpenTelemetry (OTel) — the emerging open, vendor-neutral standard for collecting metrics, logs, and traces in a unified way, now widely adopted across the industry.

4.5 Comparing the Three Pillars

Pillar	Answers	Granularity	Volume/Cost	Best For
Metrics	“What is happening, and how much?”	Aggregated	Low	Dashboards, alerting, trends
Logs	“What exactly happened, and why?”	Per-event	High	Root cause detail, debugging
Traces	“Where in the request did it happen?”	Per-request	Medium-High	Latency bottlenecks, distributed debugging

4.6 How the Pillars Work Together (Worked Example)

Scenario: Users report the checkout page is slow.

- Metrics** show p99 latency on `checkout-api` jumped from 200ms to 3.5s starting at 10:14 AM, and error rate is unchanged (so it’s a *performance* issue, not a *failure*).
- Traces** for requests during that window show the `payment-service` span consistently taking 3+ seconds, while other spans are normal.
- Logs** from `payment-service` during that window reveal repeated messages: `WARN: connection pool exhausted, waiting for available connection`.

Conclusion: The payment service’s database connection pool is exhausted — likely due to a spike in traffic or a slow downstream dependency holding connections open. This diagnosis took minutes because metrics → traces → logs were used together, each narrowing the search space.

This “narrowing funnel” (metrics to detect → traces to localize → logs to explain) is one of the most important mental models in this entire course, and directly sets up Lecture 3’s Root Cause Analysis content.

4.7 Class Exercise (10 min)

Break students into pairs. Give each pair one scenario card (e.g., “checkout errors spike,” “API latency increases only for EU users,” “database CPU pegged at 100%”). Ask them to describe which pillar (metric, log, or trace) they’d check first, second, and third, and why.

Part 5: Recap & Discussion (10 min)

Key Takeaways

1. **Infrastructure context** turns raw alerts into actionable information — it's the foundation every other ITOps capability is built on.
2. A **CMDB** is the system of record for what exists in your environment and how components relate to each other.
3. **Topology mapping** visualizes those relationships and is increasingly **dynamic and auto-discovered** in cloud-native environments, because static diagrams cannot keep up with ephemeral infrastructure.
4. **Metrics, Logs, and Traces** are the three pillars of observability — each answers a different question, and real troubleshooting uses all three together.
5. These four concepts (context, CMDB, topology, observability) are the raw materials that AI/ML models (Lecture 2) and correlation engines (Lecture 3) consume — none of the “intelligent” capabilities work without this foundation being solid first.

| Discussion Questions for Wrap-Up

1. Why might an organization with excellent monitoring tools still struggle with incident response if their CMDB is inaccurate or outdated?
2. In a Kubernetes environment where pods are recreated every few minutes, why is dynamic topology discovery non-negotiable?
3. Can you think of a situation where logs alone would be misleading without metrics or traces for context?

| Suggested Pre-Reading for Lecture 2

- Basic familiarity with what a time series is (a sequence of data points indexed by time).
- Optional: skim Google's SRE Book chapter on Monitoring Distributed Systems (freely available online) for the “four golden signals.”

End of Lecture 1 — IT Operations & Observability Foundations